

DOKUZ EYLÜL UNIVERSITY
ENGINEERING FACULTY
DEPARTMENT OF COMPUTER ENGINEERING

**AI-Supported Voice-Assisted Medication Tracking
System for Elderly Individuals and Caregivers**

by

2022510140 Esma Oruç

Advisor

Prof.Dr. Semih Utku

DECEMBER 2025

İZMİR

AI-Supported Voice-Assisted Medication Tracking System for Elderly Individuals and Caregivers

A Thesis Submitted to the

Dokuz Eylül University, Department of Computer Engineering

In Partial Fulfillment of the Requirements for the Degree of B.Sc.

by

Esma Oruç

Advisor

Prof.Dr. Semih Utku

DECEMBER 2025

İZMİR

CHAPTER ONE	3
INTRODUCTION	3
1.1 Background Information	3
1.2 Problem Definition	4
1.3 Motivation	5
1.4 Goal and Contribution	6
1.4.1 Goal	6
1.4.2 Contribution	7
1.5 Project Scope	7
Included in scope:	8
Not included (future work):	8
1.6 Standards, Ethics, Constraints and Conditions	8
CHAPTER TWO	10
LITERATURE REVIEW	10
2.1 Related Works	10
2.1.1 IoT and Hardware-Based Medication Systems	10
2.1.2 Voice-Assisted Interactions for the Elderly	10
2.1.3 Computer Vision and AI in Pharmaceutical Management	11
2.2 Comparison with the Existing Solutions	11
Discussion and Gap Analysis:	13
CHAPTER THREE	15
REQUIREMENTS	16
3.1 Functional Requirements	16

3.1.1 Authentication and Security Module	16
3.1.2 Medication Onboarding (AI & OCR)	16
3.1.3 Scheduling and Notification Engine	16
3.1.4 Intake Verification and Stock Management.....	17
3.1.5 Reporting and Monitoring.....	17
3.2 Non-Functional Requirements.....	17
3.2.1 Performance and Latency.....	17
3.2.2 Reliability and Availability	18
3.2.3 Security and Data Privacy.....	18
3.2.4 Usability and Accessibility	18
3.3 System Interfaces	18
CHAPTER FOUR.....	19
DESIGN	19
4.1 Architectural View.....	19
4.2 Database Design / ER Diagram	20
4.2.1 Core Entities: User & Medication Management.....	20
4.2.2 Scheduling Engine	21
4.2.3 Medical History & System Logs.....	22
4.3 UML Class Diagram.....	23
4.3.1 User & Role Management Module	23
4.3.2 Medication Management Module	24
4.3.3 Scheduling & Intake Tracking	25
4.3.4 Medical History Module	26

4.3.5 AI Logs & System Audit	27
4.4 UI Design.....	28
4.5 Use Case Diagram	29
4.5.1 Actors	29
4.5.2 Key Use Case Scenarios	30
4.6 Sequence Diagram	31
4.6.1 User Authentication Flow	31
4.6.2 Critical Alert & Remote Intervention	32
4.6.3 Weekly Report Generation.....	33
4.7 Activity Diagram	33
4.8 State Diagram	35
4.8.1 Medication Intake Lifecycle	35
4.8.2 AI-Assisted Onboarding State	36
4.9 Deployment Diagram.....	37
CHAPTER FIVE.....	38
IMPLEMENTATION	39
5.1 System Architecture and Project Structure	39
5.2 Data Model and Database Design.....	39
5.2.1 Users and profiles.....	40
5.2.2 Medication data.....	40
5.2.3 Adherence tracking	40
5.2.4 Care team	40
5.2.5 Stock management:	40

5.3 Authentication and Role-Based Access	41
5.4 Patient-Side Features	41
5.4.1 Home Screen	42
5.4.2 Medication Cabinet	43
5.4.3 Calendar and History	45
5.4.4 Profile and Settings	46
5.4.5 Care Team Management	47
5.5 Caregiver-Side Features.....	47
5.5.1 Caregiver Dashboard.....	47
5.5.2 Patient List	48
5.5.3 Activity Log	48
5.6 AI-Powered Features	49
5.6.1 AI Medication Note Generation.....	49
5.6.2 Camera-Based Medication Scanning	50
5.7 Internationalization and Accessibility	51
5.7.1 Language Support	51
5.7.2 Accessibility	51
REFERENCES.....	53

CHAPTER ONE

INTRODUCTION

1.1 Background Information

Medication adherence, which means taking medicine as prescribed, is an important factor that affects treatment results, especially for older adults with chronic illnesses. Studies have shown that up to 50% of older adults fail to adhere to their prescribed medication schedules, leading to serious health complications, increased hospitalizations, and unnecessary healthcare costs (Grindrod et al., 2014). For this reason, ensuring accurate and timely medication intake has become a key challenge in geriatric healthcare.

In recent years, mobile health (mHealth) applications have been developed to support medication management. These systems typically provide digital reminders, dosage tracking, and reporting features. However, existing applications have not been sufficiently designed for older adults. Researchers have identified usability problems such as small font sizes, low contrast, and complex navigation structures that reduce accessibility and discourage continuous use (Grindrod et al., 2014). As a result, many older users with vision problems or low-tech skills find it hard to use these apps.

To overcome these barriers, researchers have explored alternative interaction methods such as voice-based interfaces and smart assistants. Voice-assisted systems have been shown to improve medication adherence by allowing users to receive spoken reminders and interact verbally with the device (Shade et al., 2021). Voice technologies such as Text-to-Speech (TTS) and Speech-to-Text (STT) can bridge the accessibility gap by transforming written instructions into speech and enabling simple voice confirmations. However, most voice-based solutions remain integrated into commercial ecosystems (e.g., Alexa, Google Assistant) and are not customized for older users' needs, especially those requiring caregiver supervision (Liu et al., 2023).

Caregiver-linked systems have also been proposed to enable remote monitoring of patients' medication behavior. These approaches allow caregivers or family members to verify whether a medication has been taken and to receive alerts when doses are missed

(Liang et al., 2023). Still, most of these systems depend on manual input and are not easy for older adults to use.

Therefore, there is still a need for an accessible, voice-assisted, and caregiver-connected medication tracking system that simplifies the process for elderly users while providing reliable monitoring and feedback to caregivers. This project aims to address that gap by designing a mobile application that integrates clear visual design, audio support, and a confirmation mechanism to improve adherence and communication between patients and caregivers.

1.2 Problem Definition

Medication non-adherence remains one of the most persistent issues in elderly healthcare. Studies have indicated that nearly half of older adults do not follow their prescribed medication schedules, leading to increased hospitalization, complications, and mortality (Sokol et al., 2005; Wilson et al., 2007). The primary reason for non-adherence is not the absence of tools, but the lack of systems designed with accessibility, usability, and intelligent automation in mind.

Numerous mobile medication reminder applications have been developed to address this challenge. However, recent studies have shown that these tools often fail to meet the specific cognitive, visual, and motor needs of older adults (Stuck et al., 2017; Grindrod et al., 2014). Most existing apps contain small text, low color contrast, and complex navigation paths, which make interaction difficult.

A critical deficiency in current market solutions is the lack of AI-supported automation. Most medication apps rely heavily on manual data entry, requiring users to type out complex medication names and dosages on small screens. For elderly users with tremors, arthritis, or declining vision, this manual process is physically difficult and error prone. The absence of AI technologies, such as Optical Character Recognition (OCR), compels users to perform tedious manual tasks, leading to frustration and eventually abandonment of the application.

Furthermore, many applications assume that users are technologically literate, which is rarely the case for elderly populations. As a result, many older users abandon these tools after short-term use.

Another limitation of current systems is their inability to confirm medication intake or provide meaningful feedback to caregivers. While some applications send push notifications, they cannot verify whether the medicine has actually been taken. This creates uncertainty for both patients and their caregivers. Previous research (Shade et al., 2021) has demonstrated that voice-based reminders can enhance adherence, but the integration of such functionality with AI-driven data entry and caregiver connectivity remains limited.

There is still a clear gap between most reminder apps and what elderly users actually need. Many current tools do not offer voice help, automated AI entry, easy confirmation, or ways for caregivers to follow use remotely. Fixing these issues is important for older adults who face vision, memory, or technology difficulties. This project tries to close that gap with a simple AI-supported and voice-assisted tracking system that gives reminders, gives feedback aloud, and keeps caregivers informed in an easy-to-use way.

1.3 Motivation

The motivation for this project originates from a personal observation within my own family. My grandmother, who takes multiple medications daily, often struggles to remember which medicine to take and when. Despite her efforts to use mobile alarms or written schedules, these methods have not provided a reliable solution. Moreover, her limited familiarity with modern smartphones and her visual difficulties make it challenging to use existing mobile applications. This personal experience has highlighted a significant gap between the available technological tools and the real needs of elderly users.

Medication adherence is not only a personal issue but also a growing global problem. Many elderly people forget or skip their medications, which affects their health and daily life. In Türkiye, where families still play a big role in elderly care, most caregivers use

simple ways like notes, pillboxes, or phone calls to remind their loved ones. While helpful, these methods do not offer automation, feedback, or accessibility support for users who need extra assistance.

1.4 Goal and Contribution

1.4.1 Goal

The main goal of this project is to design and develop an AI-supported voice-assisted mobile medication tracking system that helps elderly people take their medicines on time and allows caregivers to follow their medication routines remotely. The project aims to reduce the technical and cognitive barriers that make existing systems difficult to use and improve automation through artificial intelligence.

Specific objectives:

- Use Text-to-Speech (TTS) technology to provide voice reminders for users with visual or reading difficulties.
- Design a simple and clear interface with large buttons, strong color contrast, and minimal steps.
- Allow caregivers to manage medication schedules, receive notifications, and see reports.
- Add a confirmation feature (button or voice input) to check if the medicine was taken.
- Implement AI-powered image recognition to extract medication details automatically from package labels.
- Explore AI-driven personalization, such as optimizing reminder times based on user interaction data.
- Generate daily and weekly summaries for better monitoring and long-term health tracking.

1.4.2 Contribution

This project contributes to the area of assistive and intelligent health technologies by integrating accessibility-focused design with simple artificial intelligence modules. Unlike traditional reminder systems, this project introduces AI-based automation — including image recognition for easier data entry and adaptive reminders based on user behavior.

Socially, the system supports independence and safety for elderly users while reducing the monitoring burden on caregivers. Technically, it demonstrates how voice interaction, AI automation, and user-centered design principles can be combined to create practical digital health solutions for people with low digital literacy.

1.5 Project Scope

This project involves designing and developing a voice-assisted and AI-supported mobile medication tracking system that connects elderly patients with their caregivers. It will be built using React Native (Expo) for cross-platform compatibility on Android and iOS devices.

The system will have two main interfaces:

- Patient interface: provides voice and visual reminders, allows confirmation of medication intake, and supports automatic reading of medication labels using AI-based image recognition.
- Caregiver interface: lets caregivers create schedules, upload medicine photos, and track adherence data in real time.

The backend will use a cloud-based infrastructure (e.g., Supabase) for authentication, data storage, and push notifications. A TTS module will create voice reminders, while lightweight AI components will analyze interaction data to recommend improved reminder schedules in the future.

Included in scope:

- Building a cross-platform app with React Native (Expo)
- Designing accessible interfaces for patients and caregivers

- Implementing notification, TTS, and AI-based label recognition modules
- Adding confirmation and reporting features
- Generating weekly adherence reports
- Deep-learning-based NLP for voice understanding

Not included (future work):

- Full integration with IoT smart pillboxes or wearable sensors
- Real-time connection with pharmacies or hospital systems

The first version of the system will deliver a functional prototype demonstrating One prominent example is the SPEC 2.0 system, which proposes a "smart medication management ecosystem." This system combines a sensor-equipped pillbox with an Android application to track when the box is opened and sends SMS alerts to caregivers if a dose is missed. While its automatic dosage monitoring is a technical strength, the study acknowledges a critical limitation: the system cannot verify if the medication was actually ingested after removal, and the hardware dependency creates a barrier to entry for low-income users.

medication reminders, AI-assisted data entry, and caregiver monitoring. Future versions may extend AI capabilities for smarter personalization and automation.

1.6 Standards, Ethics, Constraints and Conditions

The project follows general software engineering and accessibility standards to ensure usability for elderly users. User interface design will consider readability, contrast, and interaction simplicity in accordance with accessibility guidelines (WCAG).

All data used in the system will be treated according to ethical data privacy principles. Only essential medication information will be stored, and no personal or medical records will be collected. Any voluntary feedback from users will be obtained with informed consent.

CHAPTER TWO

LITERATURE REVIEW

2.1 Related Works

The domain of medication adherence and elderly care technology has been extensively studied, with solutions ranging from mechanical dispensers to sophisticated software applications. The existing literature can be broadly categorized into three main domains: IoT-based hardware systems, voice-assisted user interfaces, and AI-driven mobile applications.

2.1.1 IoT and Hardware-Based Medication Systems

A significant portion of early and current research focuses on the "Internet of Things" (IoT) to create smart environments for medication tracking. These systems often rely on physical hardware integrated with software to monitor adherence physically.

Similarly, Liang et al. (2023) reviewed in-home geriatric care systems utilizing robot assistants and wearable sensors. Their research indicates that while these devices support patients with polypharmacy by providing physical guidance, the high cost and complex installation processes discourage widespread adoption among the elderly. Furthermore, Paul et al. (2024) introduced a "Smart Medicine Reminder Kit" that triggers phone calls to the patient upon missed doses. Although effective for visibility, the study notes that elderly users are often reluctant to adopt cumbersome hardware devices that disrupt their daily routine.

2.1.2 Voice-Assisted Interactions for the Elderly

Recognizing the visual and motor limitations associated with aging, researchers have increasingly investigated Voice User Interfaces (VUIs) as an alternative to touchscreens.

A recent study on Voice Assistants for Health Self-Management (Mahmood et al., 2024) explored how older adults interact with AI-based voice agents. The findings

revealed that elderly users view voice assistants not merely as tools but as companions, preferring a conversational style that is slower, clearer, and more empathetic than standard commercial assistants like Alexa or Siri.

Another study focusing on Voice Usability (2025) examined the learning curve of adults over 55 using voice commands. While users learned basic commands quickly, they struggled significantly with complex navigation structures and expressed concerns over security and accent recognition. These studies collectively suggest that for a voice system to be effective in medication management, it must be specialized, simplified, and capable of "human-like" interaction rather than generic command processing.

2.1.3 Computer Vision and AI in Pharmaceutical Management

The most recent trend in literature shifts from hardware toward software-based solutions leveraging Artificial Intelligence (AI) and Computer Vision.

Vergara-Villegas et al. (2025) proposed an innovative system combining Augmented Reality (AR) with an automatic pillbox to visually guide users in identifying pills. While pilot studies found AR to be motivating, the cognitive load required to use AR glasses or complex visual overlays was identified as a potential hurdle for geriatric users with cognitive decline.

In the realm of mobile software, applications like "MedScan" (2025) have demonstrated the potential of deep learning to identify medication packaging for visually impaired users with high accuracy. Similarly, recent implementations utilizing Tesseract OCR in Android environments allow for the scanning of medicine labels to convert text to speech, thereby reducing the error-prone process of manual data entry.

It is important to distinguish the scope of AI in this project from clinical decision support. A scoping review on AI in Managing Polypharmacy (2025) highlighted the use of AI models to predict drug interactions. However, such systems raise significant ethical and safety concerns. Therefore, distinct from clinical AI advisors, the proposed system in this thesis utilizes AI strictly for visual recognition (OCR) and interaction (NLP), avoiding the risks associated with medical prescription advice.

2.2 Comparison with the Existing Solutions

The analysis of related works reveals a significant gap between high-cost hardware solutions and generic mobile applications. Existing apps often lack accessibility features, while smart hardware is often too expensive or complex.

Table 2.1 presents a detailed feature comparison between the proposed system and prevalent approaches in the literature.

Table 2.1 Comparison of the proposed system with existing medication management solutions.

	Smart Pill System	IoT Review (2024)	Voice-Usability (2025)	AR + Pillbox (2025)	In-home Geriatric (2023)	XAI Polypharmacy (2025)	Voice Management (2024)	Self-Smart Reminder Kit (2024)
Medication adherence support	X	X	General Purpose	X	X	Prediction Model		X
Elderly-friendly design	X	X(Partial)	X	X	X		X	
Voice reminders			X				X	X
Voice input / TTS/STT			X				X	
Caregiver monitoring					X			
AI-powered label recognition								

IoT device involved	X	X		X	X			X
Wearable integration		X			X			
Smart pillbox / gadget	X	X		X				X
Push notifications	X				X			X
Health monitoring		X			X			X
Remote alerts	SMS Alerts				X			X(phone call)
Home-care focus	X	X		x	X		X	X
Accessibility / large UI	X		X	Complex AR	X		X	
Cost / hardware requirement	X	X		X	X			X

Discussion and Gap Analysis:

As detailed in Table 2.1, the existing landscape of medication management creates a fragmented experience for elderly users. The comparison highlights five critical gaps that our project addresses:

1. The Hardware Barrier: As seen in the "Smart Pill System," "IoT Review (2024)," "AR + Pillbox (2025)," and "Smart Reminder Kit (2024)" columns, a significant majority of current solutions rely on external IoT devices or smart gadgets. The row "Cost / hardware requirement" indicates that these solutions entail high costs and complex setups. In contrast, our project (final column) eliminates this dependency entirely, utilizing the smartphone as the sole required device, thus ensuring low cost and easier adoption.

2. The Disconnect Between Voice and Monitoring: Studies focusing on usability, such as "Voice-Usability (2025)" and "Voice Self-Management (2024)," successfully implement elderly-friendly voice interfaces. However, as shown in the table, these studies fail to include "Caregiver monitoring" or "Remote alerts." Conversely, systems that do have monitoring (like "In-home Geriatric") lack voice interaction capabilities. Our project bridges this gap by combining the accessibility of voice input with the safety of real-time caregiver alerts.

3. Lack of Automated Data Entry (AI Label Recognition): A critical observation from Table 2.1 is that the "AI-powered label recognition" row is empty for almost all related works. While "AI Polypharmacy (2025)" utilizes AI, it is restricted to clinical analysis and does not assist the patient in daily data entry. Our project fills this specific void by implementing OCR technology, allowing users to automatically scan and log their medications without manual typing or hardware sensors.

4. Cognitive Load and Interface Complexity: The comparison reveals significant usability inconsistencies in the "Accessibility / large UI" row. As noted in the table, advanced solutions like "AR + Pillbox" present a "Complex AR" interface. While innovative, Augmented Reality introduces a high cognitive load that is often unsuitable for geriatric users with declining vision or cognitive processing speeds. Similarly, many IoT interfaces rely on small indicators rather than clear guidance. By adopting a "Large UI" and "Voice-First" approach, our proposed system minimizes these cognitive barriers, offering a simpler and more accessible experience than the complex visual overlays of AR systems.

5. Portability and Lifestyle Integration: As indicated in the "Smart pillbox / gadget" row, solutions such as the "Smart Pill System" and "Smart Reminder Kit" rely on stationary devices. This creates a "Location Dependency," meaning the medication tracking is tied to a specific physical location (usually the home). This restricts the user's mobility and lifestyle. Our project, being purely mobile-based (as shown in the final column), addresses this gap by ensuring adherence support is portable and available wherever the user goes, promoting a more independent lifestyle for the elderly.

In conclusion, while other systems excel in isolation—either offering robust hardware tracking or simple voice commands—our project is unique in offering a software-only, integrated solution that combines AI-based recognition, voice interaction, and caregiver connectivity without the burden of expensive hardware or geographical limitations.

CHAPTER THREE

REQUIREMENTS

This chapter details the functional and non-functional requirements of the "AI-Supported Voice-Assisted Medication Tracking System." These requirements define the specific behaviors, functions, and constraints of the system to ensure it meets the needs of elderly users and their caregivers.

3.1 Functional Requirements

The functional requirements are categorized into modules representing the core capabilities of the system: Authentication, Medication Onboarding (AI/OCR), Scheduling, Intake Verification, and Reporting.

3.1.1 Authentication and Security Module

This module handles user identity and role management to ensure secure access.

- **FR-AUTH-01:** The system shall allow users to register and log in using a secure email and password combination.
- **FR-AUTH-02:** The system must implement Role-Based Access Control (RBAC) to distinguish between "Patient" and "Caregiver" roles upon login, rendering the appropriate interface for each user type.
- **FR-AUTH-03:** All sensitive account actions (e.g., password changes, profile updates) must be logged in the system audit logs for security auditing purposes

3.1.2 Medication Onboarding (AI & OCR)

This module automates the data entry process using Computer Vision to reduce manual typing for elderly users.

- **FR-OCR-01:** The system shall utilize the mobile device's camera to capture images of medication packaging.

- **FR-OCR-02:** The AI engine shall process the captured image to automatically extract the Medication Name and Dosage information, minimizing the need for manual text entry.
- **FR-OCR-03:** The system shall calculate a "Confidence Score" for each OCR result. If the confidence score is below 70%, the system shall trigger a "Manual Review" workflow requiring user verification.
- **FR-OCR-04:** Users shall have the ability to manually edit, correct, or confirm the AI-extracted data before saving the medication record to the database.

3.1.3 Scheduling and Notification Engine

This module ensures timely reminders are delivered reliably.

- **FR-SCH-01:** The system shall allow users (or caregivers) to define flexible medication schedules (e.g., daily, weekly, specific intervals).
- **FR-SCH-02:** The system shall trigger local push notifications and audio alarms (Text-to-Speech) at the scheduled time. These alarms must function even if the device is currently offline (no internet connection).
- **FR-SCH-03:** The notification system shall support an Escalation Protocol: If a patient does not respond to a medication alarm within 30 minutes, a Critical Alert notification must be automatically sent to the assigned Caregiver.

3.1.4 Intake Verification and Stock Management

This module tracks adherence and inventory.

- **FR-INTAKE-01:** Patients shall be able to confirm medication intake via voice commands (e.g., "I took my medicine") using Natural Language Processing (NLP) or by tapping a large confirmation button.
- **FR-INTAKE-02:** Upon successful confirmation of intake, the system shall automatically decrement the current stock value of the respective medication in the database.

- **FR-INTAKE-03:** The system shall automatically trigger a "Low Stock Warning" notification to the user and caregiver when the remaining pill quantity falls below the defined restock threshold.

3.1.5 Reporting and Monitoring

This module provides visibility into the patient's health behavior.

- **FR-REP-01:** The system shall generate weekly adherence reports, visualizing the number of "Taken," "Missed," and "Late" dosages.
- **FR-REP-02:** Authorized Caregivers shall have read-only access to the patient's history logs (including LabelScanSession and IntakeEvent) to verify daily adherence remotely.

3.2 Non-Functional Requirements

These requirements define the quality attributes, performance constraints, and reliability standards of the system.

3.2.1 Performance and Latency

- **NFR-PERF-01:** The application launch time shall not exceed 2 seconds on standard mid-range mobile devices to ensure a responsive user experience.
- **NFR-PERF-02:** The OCR processing and API response time for label scanning shall be completed in under 3 seconds over a standard 4G LTE connection.

3.2.2 Reliability and Availability

- **NFR-REL-01 (Offline First Architecture):** The system shall be designed to function in offline mode. Critical actions, such as receiving alarms and confirming intake, must be processed locally. Data recorded offline must be synchronized with the backend server automatically once network connectivity is restored.

- **NFR-REL-02:** The backend API shall maintain a target uptime of 99.9% to ensure that critical alerts and synchronization processes are available when needed.

3.2.3 Security and Data Privacy

- **NFR-SEC-01:** All data transmission between the mobile client and the backend server shall be encrypted using industry-standard security protocols (e.g., HTTPS/SSL).
- **NFR-SEC-02:** User passwords shall never be stored in plain text; they must be securely hashed and salted using a strong cryptographic algorithm before storage in the database.
- **NFR-SEC-03:** The system shall implement strict access control mechanisms to ensure that sensitive patient health data is accessible only to the authenticated patient and their authorized caregiver.

3.2.4 Usability and Accessibility

- **NFR-UI-01:** The user interface for the Patient role shall strictly adhere to accessibility guidelines (WCAG), featuring high-contrast text, large touch targets, and simplified navigation to accommodate elderly users.
- **NFR-UI-02:** The system shall provide auditory feedback (Voice Text-to-Speech) for critical actions to assist visually impaired users.

3.3 System Interfaces

- **Hardware Interfaces:** The mobile application requires permission to access the device's Camera (for OCR scanning) and Microphone (for voice commands).
- **Software Interfaces:** The system integrates with external AI services for image processing and standard mobile Notification Services (APNs/FCM) for delivering alerts.

CHAPTER FOUR

DESIGN

This chapter presents the comprehensive architectural and detailed design of the AI-Supported Medication Tracking System. It covers the high-level system architecture, database schema, object-oriented design patterns, and user interface mockups, illustrating the system's workflow from backend infrastructure to frontend accessibility.

4.1 Architectural View

The system utilizes a modern, distributed Client-Server Architecture deployed on a Virtual Private Server (VPS) infrastructure. The architecture is designed to ensure scalability, type safety, and efficient data handling using a containerized approach.

As illustrated in Figure 4.1, the system consists of the following core layers:

1. **The Client Layer (Mobile Application):** Developed using React Native (Expo) and TypeScript, this layer serves as the primary interface for Patients and Caregivers. It ensures type safety across the entire application and communicates with the backend via secure API endpoints.
2. **The Backend Infrastructure:** The backend is hosted on a Hetzner VPS, orchestrated using Coolify and Docker containers. This infrastructure hosts:
 - a. **Application Server:** Built with Node.js, Express, and tRPC, ensuring high-performance API handling.
 - b. **Database Access:** Prisma ORM is utilized to interact with the PostgreSQL database, providing strictly typed database queries and schema migrations.
3. **Data Layer:** A PostgreSQL database serves as the persistent storage for user profiles, medication logs, and schedules.
4. **External Integrations:** The system integrates with Large Language Models (LLMs) such as ChatGPT/Gemini via the Vercel SDK for intelligent processing of voice commands and OCR text correction. Network traffic is secured and accelerated through Cloudflare.

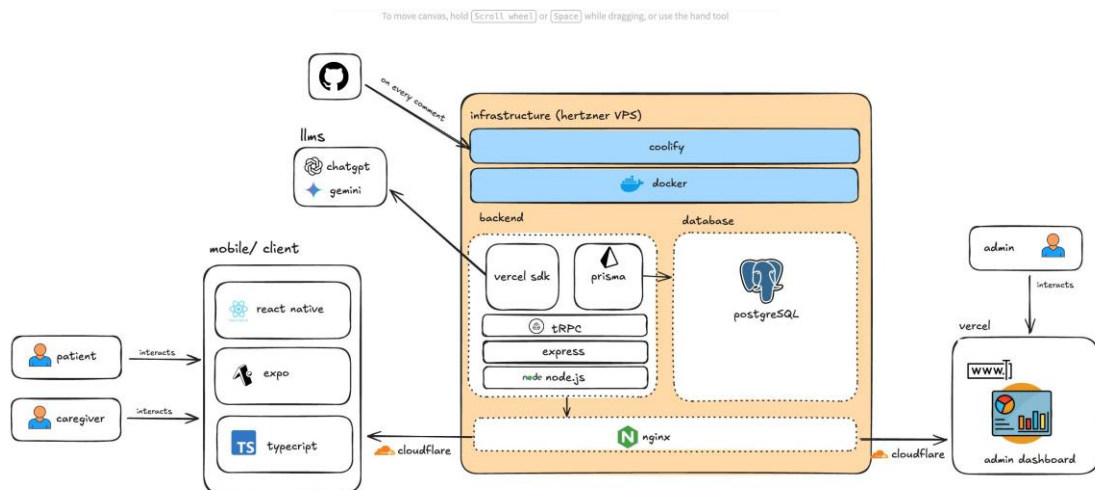


Figure 4.1 High Level System Architecture

4.2 Database Design / ER Diagram

To ensure data integrity, scalability, and efficient querying, the system utilizes a Relational Database Management System (PostgreSQL). The database schema has been meticulously designed following Third Normal Form (3NF) principles to minimize data redundancy and maintain consistency across the entire system. The logical structure is visualized across three detailed diagrams (Figures 4.2.1 – 4.2.3).

4.2.1 Core Entities: User & Medication Management

The foundation of the system relies on the interplay between user roles and medication inventory, as illustrated in Figure 4.2.1. The users table acts as the central identity entity, storing authentication credentials and common profile data. Role-specific attributes are decoupled into separate `patient_profiles` and `caregiver_profiles` tables. Simultaneously, the medication logic separates general drug information (`medications`) from specific prescriptions (`prescriptions`). These are linked via the `prescription_medications` table, which tracks critical inventory data such as `current_stock` to enable automated low-stock warnings.

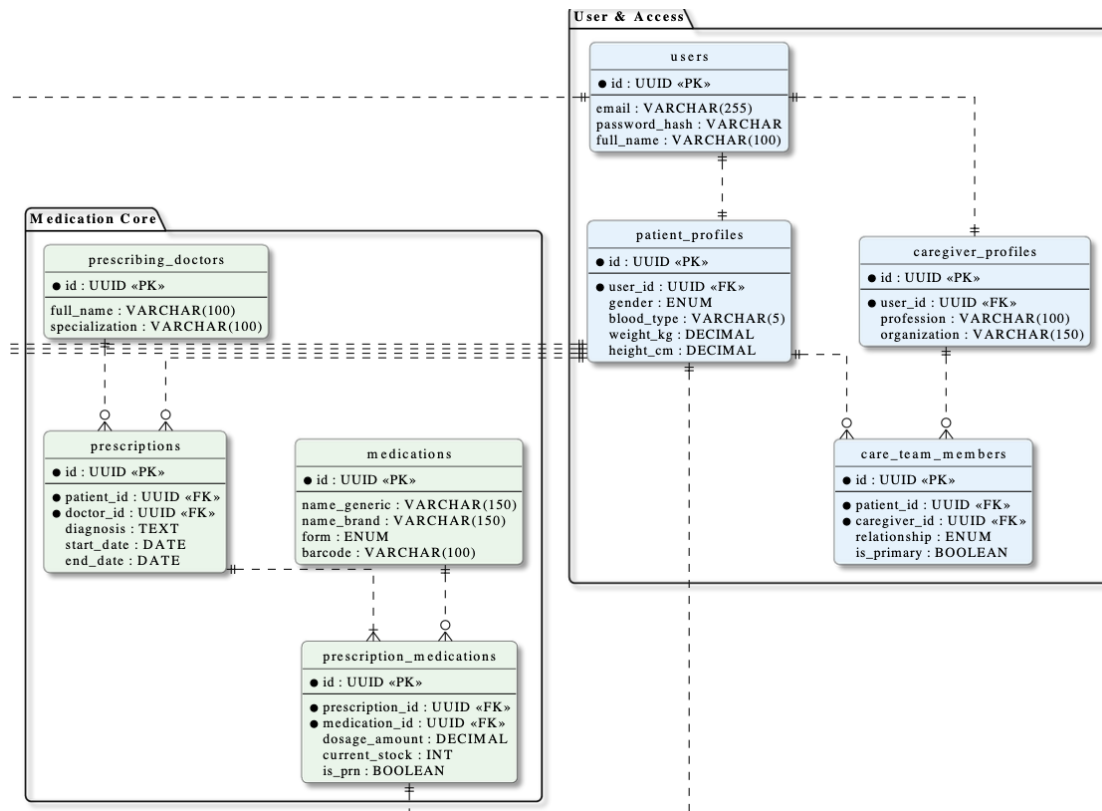


Figure 4.2.1: User Management and Medication Core Entity Relationships.

4.2.2 Scheduling Engine

The temporal aspects of the system are managed by the Scheduling Engine, detailed in Figure 4.2.2. The `dose_schedules` table defines the recurring rules (e.g., "Daily at 09:00", "Weekly"), while the `reminder_instances` table tracks the specific alarm events generated for each day. This separation allows the system to handle complex recurring patterns and track the status of every single dose independently.

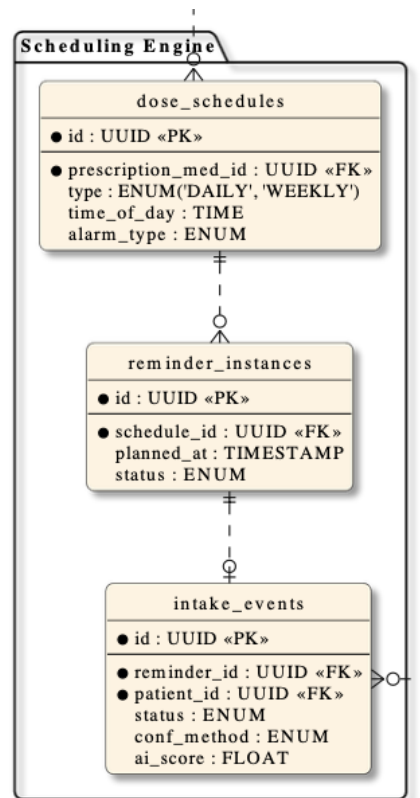


Figure 4.2.2: Scheduling Engine and Reminder Logic Schema.

4.2.3 Medical History & System Logs

Finally, patient health context and system security are managed through the module shown in Figure 4.2.3. Patient history is maintained via `chronic_conditions` and `allergies` tables to support clinical context. To satisfy security requirements (NFR-SEC-03), all critical actions are recorded in `system_audit_logs`, while AI-driven operations are tracked in `label_scan_sessions`, storing raw OCR data for debugging and model performance analysis.

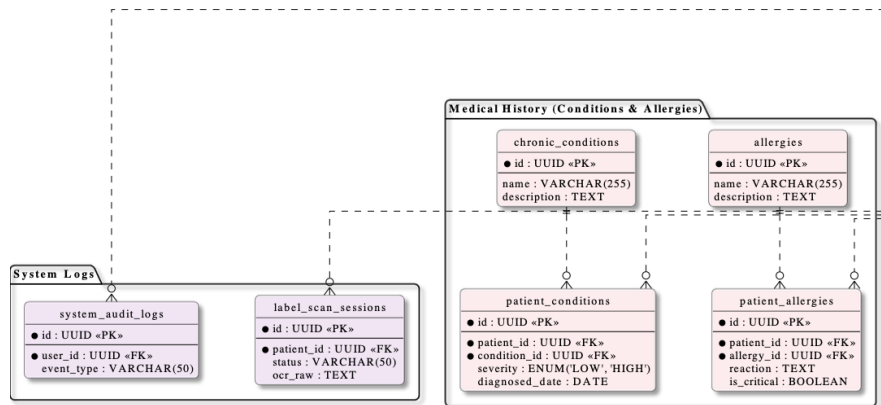


Figure 4.2.3: Medical History, Allergies, and System Audit Logs Schema.

4.3 UML Class Diagram

The system's internal logic and object structure are modeled using UML Class Diagrams. This design follows Object-Oriented Programming (OOP) principles, utilizing inheritance and composition to ensure modularity and maintainability. The class architecture is organized into five logical packages, mirroring the database schema but including business logic and data types.

4.3.1 User & Role Management Module

The user architecture, illustrated in Figure 4.3.1, utilizes an inheritance strategy to manage different user roles. The abstract User class holds common credentials (email, password hash), which are extended by concrete PatientProfile and CaregiverProfile classes. The CareTeamMember class manages the many-to-many relationship, defining specific permissions (e.g., can_edit_medication) for caregivers managing a patient.

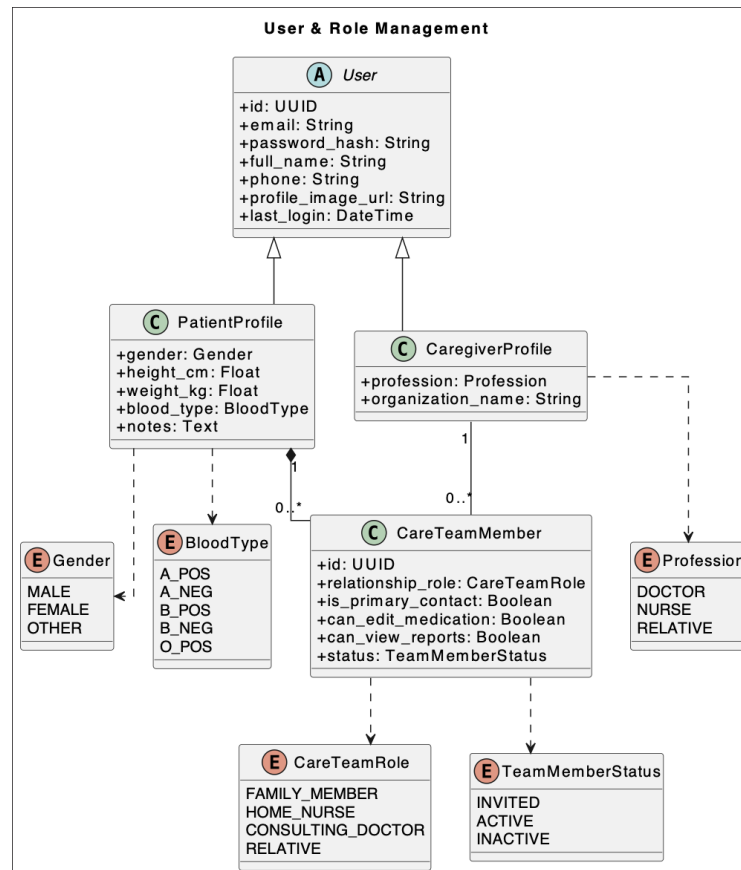


Figure 4.3.1: Class diagram for User hierarchy and Care Team roles.

4.3.2 Medication Management Module

As shown in Figure 4.3.2, the system decouples the general drug catalog from specific patient prescriptions. The Medication class defines static drug attributes (form, manufacturer), while the Prescription class holds the doctor's instructions. The PrescriptionMedication class acts as the bridge, tracking dynamic inventory data such as `current_stock` and linking to the specific administration route.

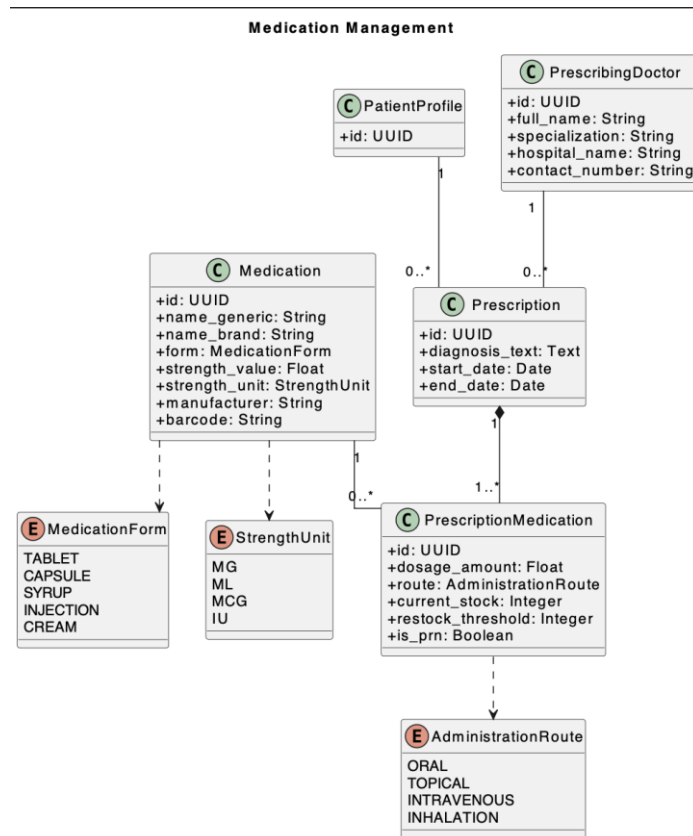


Figure 4.3.2: Class structure for Medications and Prescriptions.

4.3.3 Scheduling & Intake Tracking

The core timing logic is defined in Figure 4.3.3. The DoseSchedule class defines the recurrence rules (Daily, Weekly) and alarm preferences. The system generates ReminderInstance objects for specific alarm times. Finally, the IntakeEvent class records the actual patient action, capturing the confirmation_method (e.g., Voice Command, Photo OCR) and the AI's confidence score (ai_voice_confidence_score).

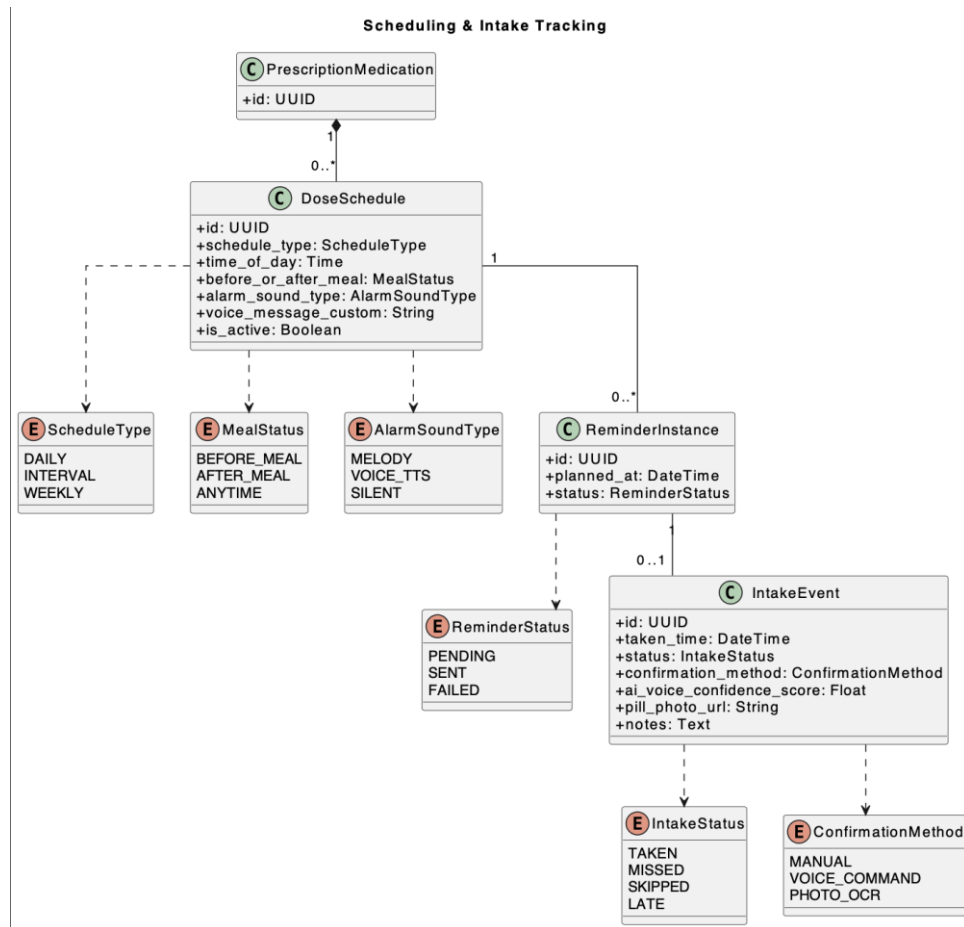


Figure 4.3.3: Scheduling engine and Intake verification logic.

4.3.4 Medical History Module

Patient background context is modeled in Figure 4.3.4. The system uses *ChronicCondition* and *Allergy* classes as reference entities. These are linked to the *PatientProfile* via intermediate classes (*PatientCondition*, *PatientAllergy*) which store specific details such as severity levels and diagnosis dates.

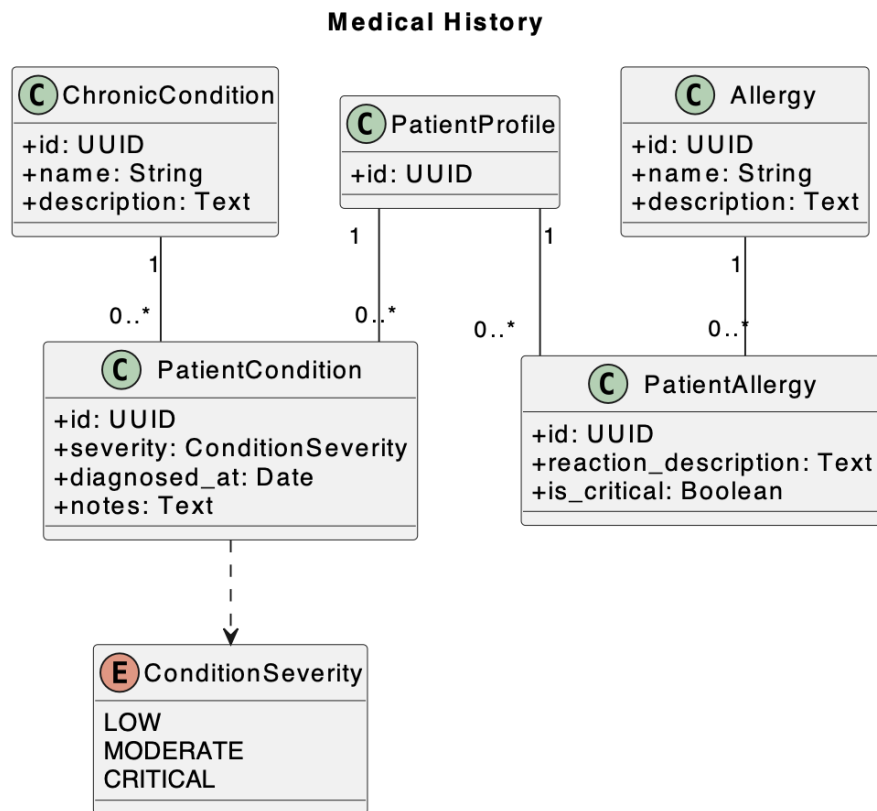


Figure 4.3.4: Medical History and Allergy tracking structure.

4.3.5 AI Logs & System Audit

To ensure security and traceability, the system implements the logging structure shown in Figure 4.3.5. The **SystemAuditLog** class records critical security events (Login, API Errors). Furthermore, the **LabelScanSession** class is specifically designed to track OCR operations, storing the **ocr_raw_text** and **parsed_data** JSON to monitor the performance of the AI recognition module.

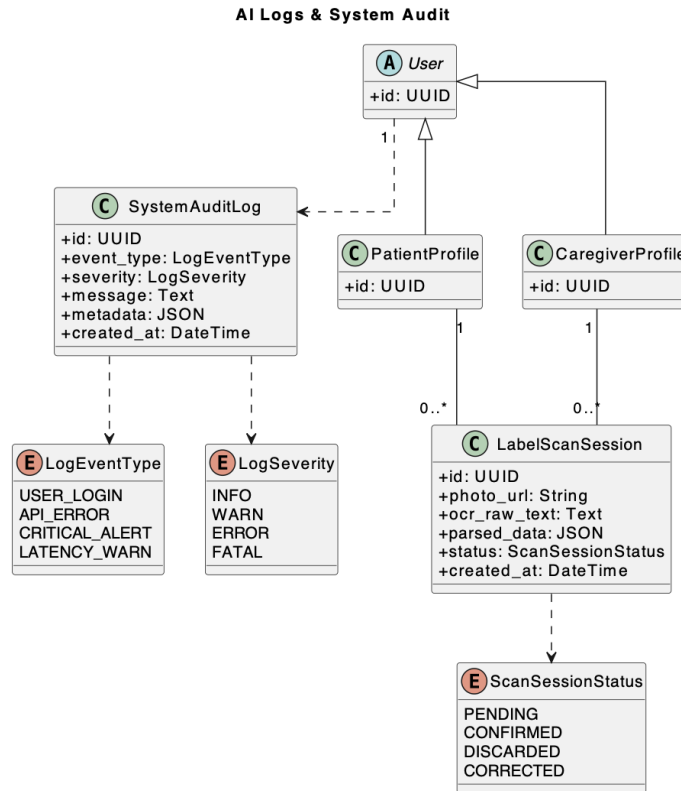


Figure 4.3.5: System Audit Logs and AI Session tracking.

4.4 UI Design

The User Interface (UI) of the system adheres to a "Patient-First" and "Accessibility-Centric" design philosophy, specifically tailored to accommodate the cognitive and physical limitations of elderly users. The design process was guided by the Web Content Accessibility Guidelines (WCAG) to ensure high contrast, readability, and ease of navigation (as defined in NFR-UI-01).

The visual design is divided into four distinct modules, as illustrated in **Figure 4.4**:

1. **Role-Based Login Screen:** To prevent confusion, the entry screen avoids complex forms. Instead, it features two large, visually distinct cards with illustrations ("I am a Patient" vs. "I am a Caregiver"), allowing users to select their role with a single tap.

2. **Elderly Patient Dashboard:** The patient interface is intentionally minimalist. It removes all unnecessary navigation elements, focusing on a single, primary action: a large, pulsing "Take Medicine" button. This design reduces cognitive load and prevents accidental clicks.
3. **AI-Assisted OCR Interface:** The medication scanning screen includes a camera overlay with a clear scanning frame. Crucially, it provides real-time feedback to the user (e.g., "We found this: Aspirin") and displays a "Confidence Score" badge, reassuring the user that the AI has correctly identified the drug.
4. **Caregiver Monitoring Dashboard:** In contrast to the patient view, the caregiver interface is data-dense. It functions as a comprehensive dashboard featuring adherence charts, critical alert logs (e.g., "Missed Dose"), and a list of managed patients, enabling efficient remote monitoring.

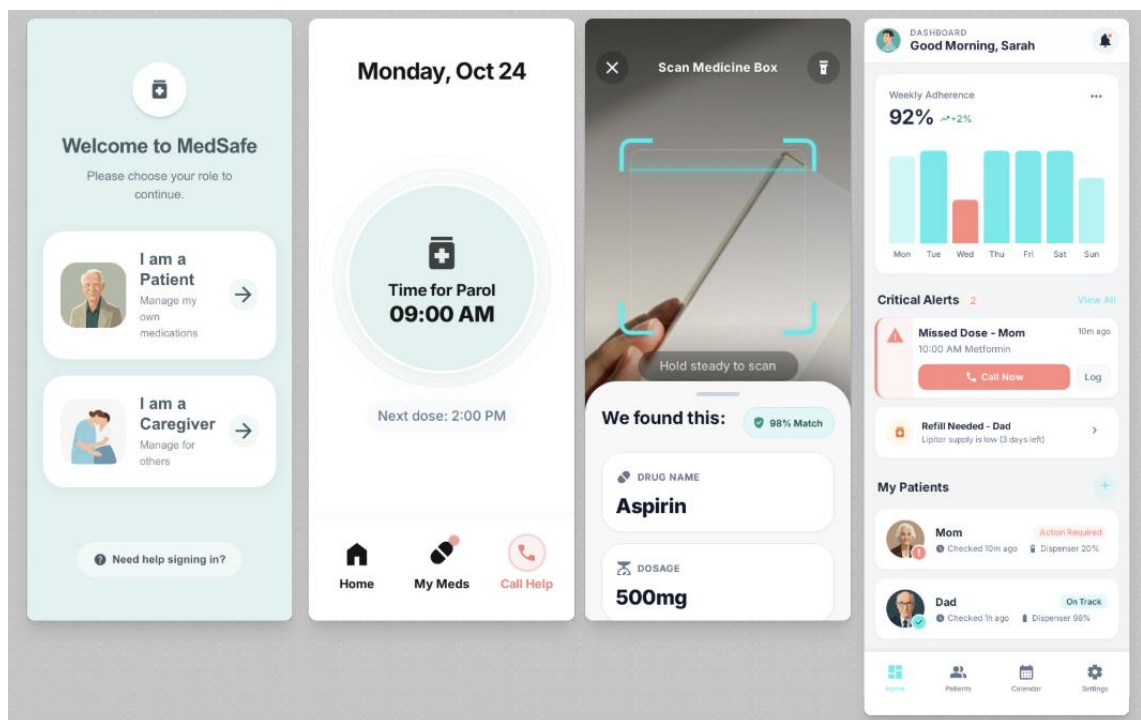


Figure 4.4: High-fidelity UI mockups showcasing the Role Selection, Patient Home, AI OCR Scanner, and Caregiver Dashboard.

4.5 Use Case Diagram

The functional requirements and user interactions are modeled using a UML Use Case Diagram. This diagram visualizes the high-level behavior of the system, defining the actors (users and external systems) and the specific use cases (functionalities) they interact with.

As illustrated in Figure 4.5, the system involves two primary human actors and three external service actors:

4.5.1 Actors

- **Elderly Patient:** The primary user of the system. Their interactions are focused on receiving reminders, confirming medication intake via voice or touch, and managing their daily schedule.
- **Caregiver:** The secondary user responsible for remote monitoring. They can view adherence reports, receive critical alerts, and assist the patient in managing medication lists.
- **Base User:** An abstract actor representing the shared functionalities (e.g., "Authenticate", "Emergency Call") common to both patients and caregivers.
- **System Actors:**
 - **AI Service:** Represents the external AI provider (e.g., OpenAI/Google Vision) used for processing OCR images and analyzing voice intents.
 - **Notification Service:** Handles the delivery of push notifications (FCM/APNS).
 - **System Timer:** An internal trigger that initiates scheduled reminders automatically.

4.5.2 Key Use Case Scenarios

1. **Medication Onboarding (Add via OCR):** When a user triggers the "Manage Medication" use case, the system includes the "Add via OCR" function. This prompts the AI Service to analyze the captured image and extract drug details automatically.

Figure 4.6.1 illustrates the secure login process. When a user submits their credentials, the Auth Service verifies the password hash against the PostgreSQL database. Upon successful validation, the system generates a JSON Web Token (JWT) containing the user's role. Based on this role, the application dynamically navigates the user to either the "Patient Home" or the "Caregiver Dashboard."

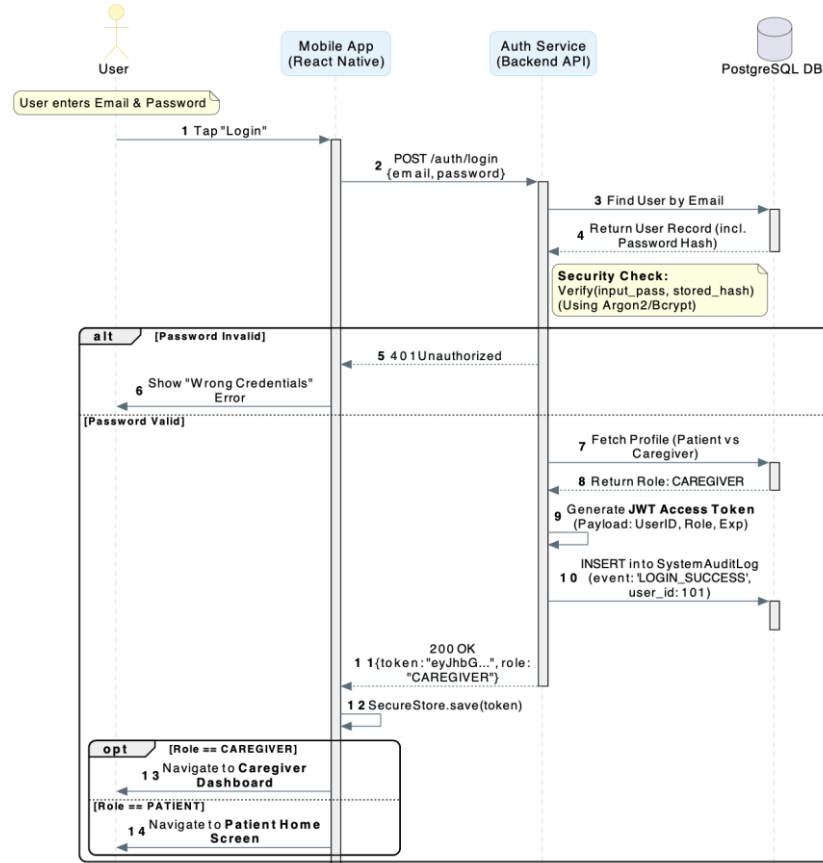


Figure 4.6.1: Sequence of the User Authentication and Role-Based Routing process.

4.6.2 Critical Alert & Remote Intervention

The system's safety net is depicted in Figure 4.6.2. This workflow is triggered automatically when a "Dose Schedule Time" threshold is exceeded without confirmation. The backend queries the database, detects the "Missed" status, and immediately dispatches a critical payload to the Notification Service (FCM/APNS). The diagram further shows the "Remote Intervention" capability, where a caregiver can initiate a VoIP call and manually mark the medication as "Taken" on behalf of the patient.

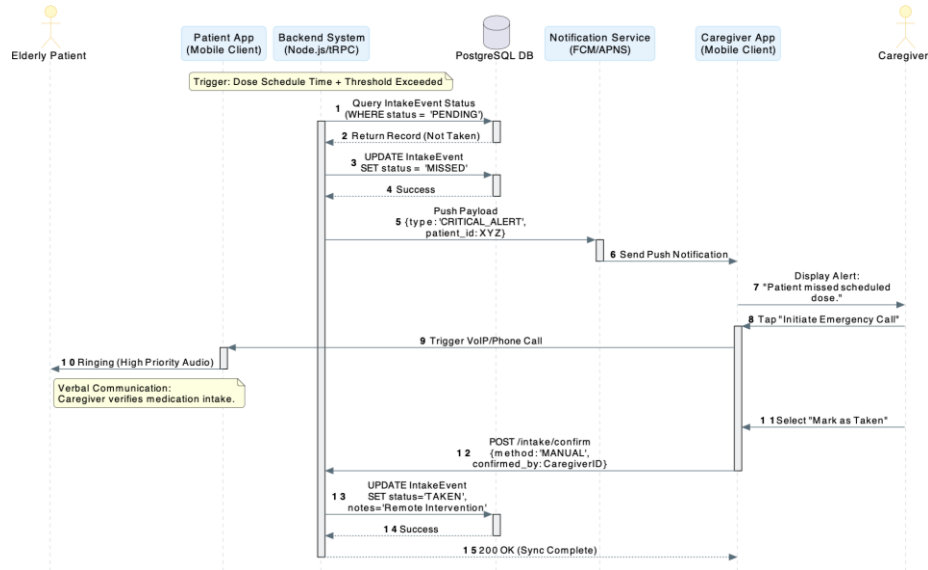


Figure 4.6.2: Workflow for Critical Missed Dose Alerts and Caregiver Intervention.

4.6.3 Weekly Report Generation

Figure 4.6.3 demonstrates how the system transforms raw data into actionable insights. When a caregiver requests a report, the Report Service aggregates data from both Intake Events (adherence logs) and LabelScanSessions (OCR logs). The AI Analytics Engine processes this data to identify patterns (e.g., "Misses morning doses often") and calculates an Adherence Score, which is finally returned to the client to be rendered as visual charts.

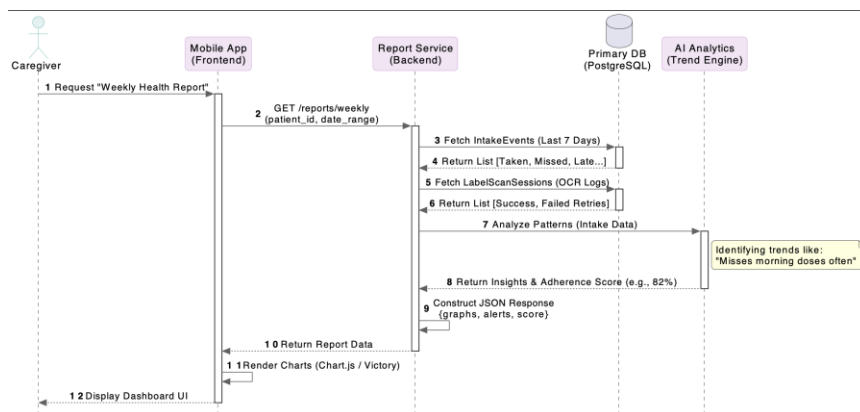


Figure 4.6.3: Data flow for generating AI-enhanced Weekly Adherence Reports.

4.7 Activity Diagram

To capture the procedural logic and business rules of the system's most complex feature, an Activity Diagram with swimlanes is utilized. This diagram specifically illustrates the "AI-Assisted Medication Onboarding" workflow, highlighting the interactions between the User, the Mobile Application, the Backend Server, and the external AI Service.

As shown in Figure 4.7, the process is divided into four distinct partitions (swimlanes):

1. **Elderly Patient:** Initiates the process by tapping "Add Medication" and interacts with the physical world (capturing the photo) and the final verification steps.
2. **Mobile App (Frontend):** Handles the hardware integration (Camera launch, image compression) and presents the UI for confirmation.
3. **Backend System:** Acts as the orchestrator, receiving the raw image, forwarding it to the AI service, and finally persisting the validated medication record and generating the schedule (Cron jobs).
4. **AI Service (OCR):** The intelligence unit that processes the computer vision tasks. It extracts the raw text, identifies the Drug Name and Dosage, and calculates a Confidence Score to determine if manual correction is needed.

The workflow includes a critical decision node ("Is Info Correct?"), allowing the user to manually intervene if the AI's prediction is inaccurate, ensuring data safety.

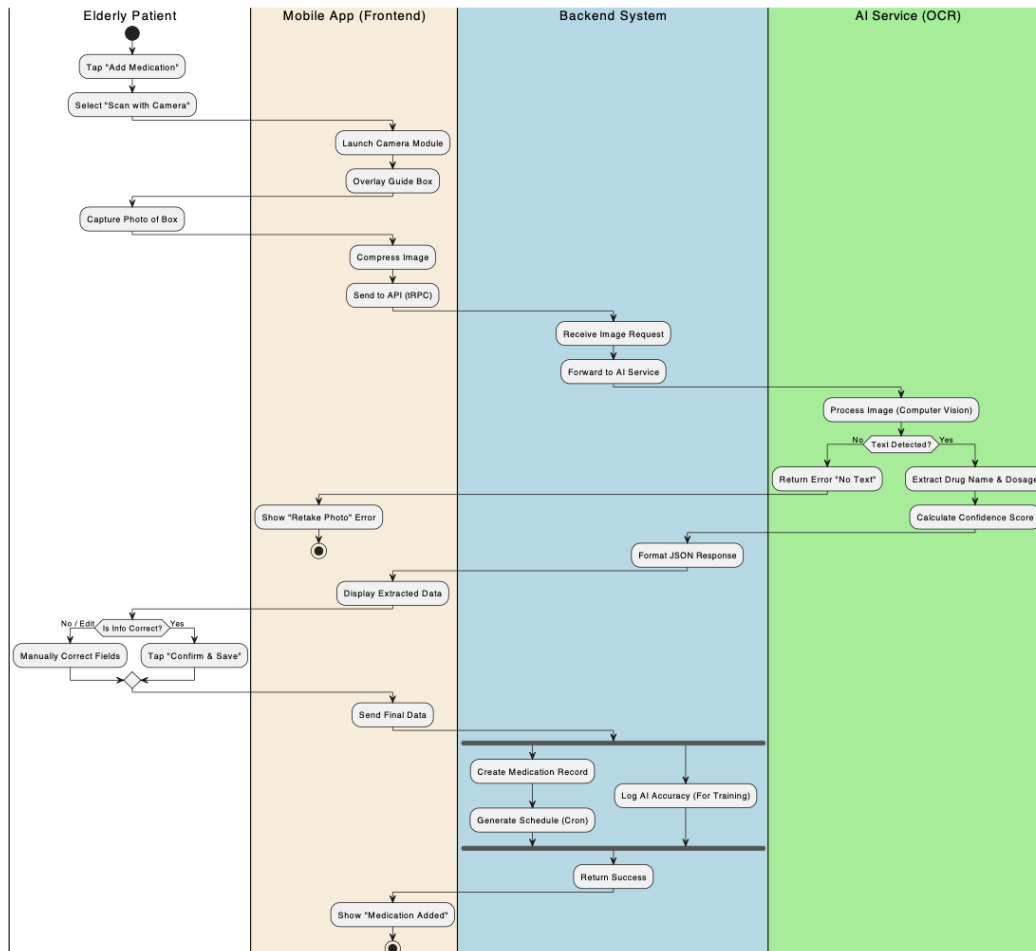


Figure 4.7: Swimlane Activity Diagram for the AI-based medication scanning and recording process.

4.8 State Diagram

While the Activity Diagram defines the flow of the system, the State Transition Diagram details the lifecycle of specific objects, illustrating how they transition from one state to another based on events and conditions. We have modeled two critical state machines for the system.

4.8.1 Medication Intake Lifecycle

Figure 4.8.1 illustrates the lifecycle of a single "Dose" object. It begins in the Scheduled state. When the time is reached, it transitions to Active Alert. The user can trigger a Snoozed state or proceed to intake. A unique feature is the AI Verification state, where the system analyzes the voice or image. Depending on the Confidence Score, the dose transitions to either Taken (if >80%) or Manual Review (if <80%). Usually, if no action is taken, the final state resolves to Missed.

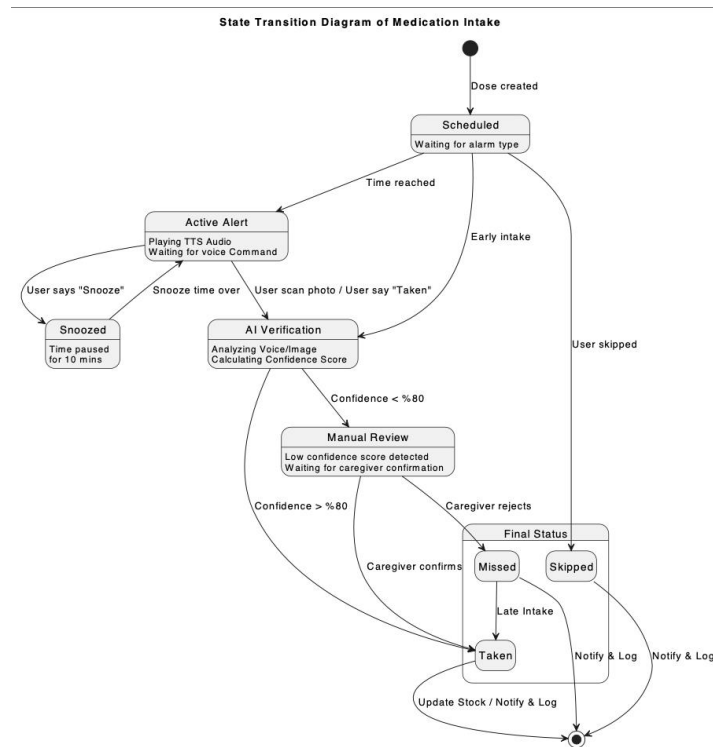


Figure 4.8.1: State machine diagram representing the lifecycle of a medication reminder.

4.8.2 AI-Assisted Onboarding State

The second diagram (Figure 4.8.2) focuses on the "Medication Creation" process via OCR. The system starts in Camera/Image Acquisition. After the capture, it enters the OCR Processing state. Based on the API response, it can transition to an Error State (if network fails) or User Verification. Critical decision nodes allow the user to switch to Manual Correction if the AI's confidence score is below 70%, ensuring data accuracy before reaching the final Persisting Data state.

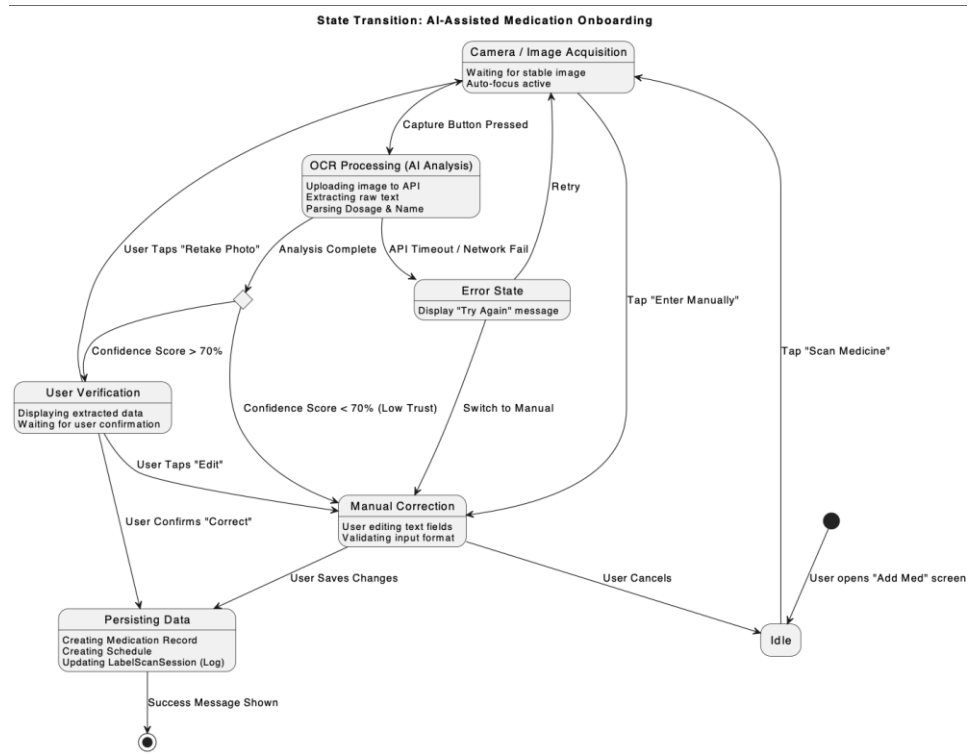


Figure 4.8.2: State transitions during the AI-based medication onboarding process.

4.9 Deployment Diagram

The physical hardware and software execution environments are detailed in the UML Deployment Diagram. Unlike traditional serverless setups, this project utilizes a robust Self-Hosted architecture deployed on a Hetzner Cloud VPS (Virtual Private Server) to ensure full data sovereignty and cost-efficiency.

As illustrated in Figure 4.9, the infrastructure is orchestrated using Coolify (an open-source PaaS), which manages the lifecycle of Docker Containers. The system comprises three main execution nodes:

1. **Client Node:** The React Native application runs on the user's mobile device. It communicates over secure HTTPS/WSS (Port 443) channels and utilizes SQLite/AsyncStore for the "Offline-First" capability.

2. **Server Node (Hetzner VPS):** This node hosts the containerized backend services:
- a. App Container: Runs the Node.js/Express backend.
 - b. DB Container: Hosts the PostgreSQL database (Port 5432).
 - c. Cache Container: Runs Redis (Port 6379) for high-speed session management and queue handling.
3. **DevOps Pipeline:** The diagram also highlights the CI/CD (Continuous Deployment) flow. Code pushed to the GitHub Repository triggers a webhook in Coolify, which automatically rebuilds and redeploys the Docker containers without manual intervention.

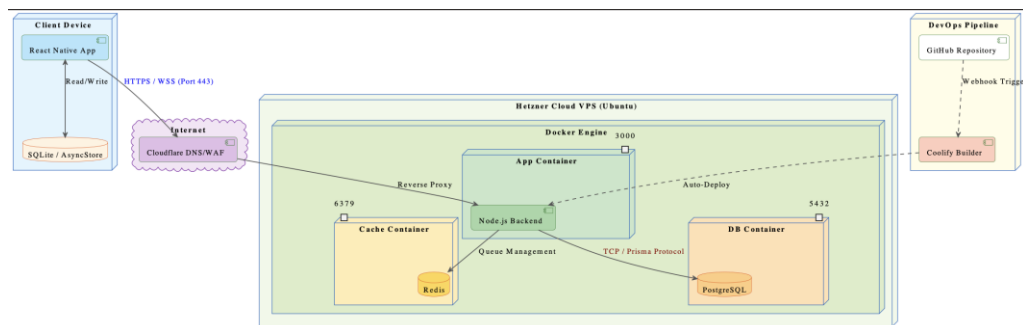


Figure 4.9: Deployment Diagram illustrating the containerized infrastructure on Hetzner VPS managed by Coolify.

CHAPTER FIVE

IMPLEMENTATION

This chapter presents the implementation of the project, the medication management system whose requirements and design were discussed in the previous chapters. The goal here is to explain how the system was actually built—what technologies were chosen and why, how the data is structured, and how each feature works from a technical standpoint.

5.1 System Architecture and Project Structure

One of the first decisions in the project was how to organize the codebase. Because the application has both a mobile frontend and an API backend, and both are written in TypeScript, it made sense to keep them in a single repository rather than splitting them into two. This is commonly referred to as a monorepo approach.

The repository is managed by Turborepo, which handles running build and development commands across the different packages in parallel. The workspace is split into three main parts:

- apps/mobile — the React Native application built with Expo
- apps/backend — the API server built with Hono and tRPC
- packages/typescript-config — shared TypeScript compiler settings

The main advantage of this layout is type sharing. The backend defines its API through tRPC routers, and the mobile app imports the router type directly from the backend package. This means that whenever a backend procedure changes its input or output shape, the frontend immediately sees a compiler error if it is using the old shape. There is no need to manually keep API contracts in sync or generate client code from an OpenAPI spec, for instance.

For day-to-day development, running `pnpm dev` at the root starts both the Expo dev server and the backend with hot-reload. The backend uses `esbuild` to bundle the TypeScript source and then runs the compiled JavaScript with Node's built-in `--watch` flag.

Both the frontend and backend organize their code by feature rather than by technical layer. In the backend, this means there are directories like `features/medication`, `features/care-team`, `features/auth`, and `features/user`, each containing the tRPC route definitions and any helper modules relevant to that domain. On the mobile side, there is a similar pattern: screens live under route groups like `(patient)` and `(caregiver)`, and reusable logic is extracted into custom hooks such as `useMedicationSchedule`, `useDashboardData`, and `useAiNote`.

5.2 Data Model and Database Design

The application stores its data in a PostgreSQL database. For local development, PostgreSQL runs inside a Docker container defined in a Compose file, which makes it easy to spin up and tear down without installing anything system-wide. Schema management and querying are handled through Prisma ORM, which generates TypeScript types from the schema and provides a type-safe query builder.

The data model can be broken into a few logical areas.

5.2.1 Users and profiles

There is a single `User` table that holds the authentication identity (name, email, verification status). Attached to each user are optional one-to-one profiles: `PatientProfile` stores patient-specific fields like birth date and medical history, while `CaregiverProfile` stores fields like specialty and institution. A user picks their role during onboarding, and the corresponding profile is created at that point. This design allows the authentication layer to remain role-agnostic —role logic is handled at the profile level instead.

5.2.2 Medication data

The `Medication` table holds canonical drug records with a generic name and an optional brand name. When a patient adds a medication to their cabinet, a `PrescriptionMedication` record is created that links the drug to the patient and carries all the dosage-specific information: amount (e.g. "500 mg"), frequency (daily, weekly, as needed, or periodic),

form (tablet, capsule, syrup, etc.), and meal timing (before meal, after meal, with food, or any). Each prescription medication can have multiple DoseSchedule entries that specify exact times of day in 24-hour format, like "08:00" or "20:00".

5.2.3 Adherence tracking

Every time a patient confirms or skips a dose, an IntakeEvent record is written with a status of TAKEN, SKIPPED, or MISSED, along with a flag indicating whether the action happened within 30 minutes of the scheduled time. These events are the basis for the adherence statistics shown on the dashboard and in the caregiver's activity log.

5.2.4 Care team

The CareTeamMember table models the relationship between patients and caregivers. It has a status field that transitions through INVITED → ACTIVE (or REJECTED). Either side can initiate the invitation—an initiatedBy field tracks whether it was the patient or the caregiver who started the connection. A unique constraint on the patient-caregiver pair prevents duplicate relationships.

5.2.5 Stock management:

Each PrescriptionMedication record also tracks currentStock (how many pills the patient has left) and restockThreshold (the point at which a low-stock warning should appear). When a dose is confirmed as TAKEN, the stock is decremented in the same database transaction that creates the intake event.

In addition to locally stored data, the system also pulls information from the OpenFDA Drug Label API at runtime to enrich medication notes, but that is covered in Section 5.6.

5.3 Authentication and Role-Based Access

Authentication is implemented using the Better Auth library, which provides email-and-password sign-up and sign-in out of the box. On the backend, it plugs into Hono through a catch-all route at /api/auth/*, and it uses Prisma as its database adapter, so

session and account records live in the same PostgreSQL database as all other application data.

On the mobile side, sessions are persisted using Expo SecureStore, which is an encrypted key-value store provided by the Expo SDK. This means that after a user logs in, they stay logged in across app restarts without their session token being stored in plain text.

After a new user signs up, they are automatically signed in and taken to a role-selection screen where they choose whether they are a patient or a caregiver. The backend's `setupRole` procedure creates the corresponding profile record. Once the role is set, it is permanent for that account.

The root layout of the app has a routing guard that checks the user's authentication and role status on every render. The logic is roughly:

- If the user is not authenticated, redirect to the login screen.
- If the user is authenticated but has no role, redirect to role selection.
- If the user is a patient, redirect to the patient tab navigator.
- If the user is a caregiver, redirect to the caregiver tab navigator.

This is implemented as a `useEffect` in the root layout component, and it uses `setTimeout(..., 0)` to avoid a React Navigation warning about navigating before the navigator is fully mounted. It is a bit of a workaround, but it reliably avoids the race condition.

5.4 Patient-Side Features

The patient experience is organized around four tabs: Home, Calendar, Meds (the medication cabinet), and Profile.

5.4.1 Home Screen

The home screen is meant to give the patient a quick snapshot of their day. It is built using a custom `useDashboardData` hook that composes several smaller hooks together—

useAdherence for the daily adherence stats, useMedicationSchedule for computing the timeline, and usePatientInvites for pending care team invitations.

At the top, an adherence summary card shows how many of today's scheduled doses have been taken, displayed as both a fraction and a percentage. Below that is the "Up Next" card, which highlights whichever dose is coming up soonest. This card has a "Take Now" button that lets the patient confirm their dose without leaving the home screen.

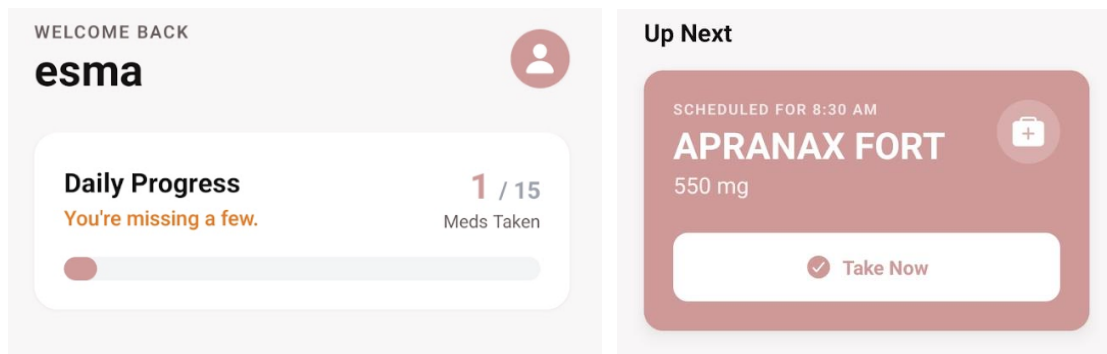


Figure 5.1: Patient dashboard

The rest of the screen is a timeline of today's scheduled medications. Each item shows the drug name, dosage, scheduled time, and its current status (taken, missed, or upcoming). At the bottom, there is a separate section for medications with an "as needed" frequency, since those do not have fixed scheduled times.

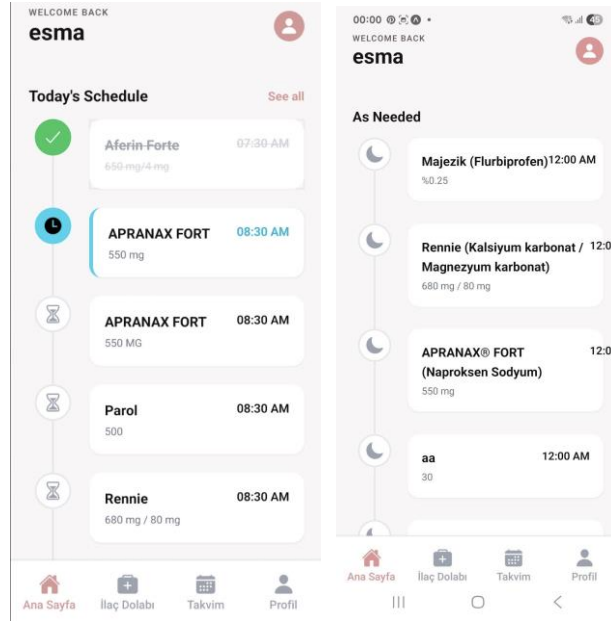


Figure 5.2: Patient daily medication list

If the patient has any pending care team invitations from caregivers, those appear as dismissible cards between the adherence summary and the timeline.

5.4.2 Medication Cabinet

The Meds tab lists all of the patient's active medications using a FlatList. Each card shows the drug name (brand name if available, otherwise generic), dosage amount, frequency, and remaining stock count as a badge.



Figure 5.3: Medication cabinet page

There are two ways to add a new medication: manually through a multi-step form, or by scanning the medication box with the phone's camera (see Section 5.6.2 for how the scanning works). The user chooses between these options from an alert dialog that appears when they tap the "+" button.

The figure displays three sequential screenshots of the 'New Medication' form, illustrating the multi-step process for adding a new medication.

- Screen 1: Medication Details**
 - Header: New Medication (with a back arrow and a red '+' button icon).
 - Instruction: Add a new medication to your schedule.
 - Section: Medication Details
 - BRAND NAME: Input field with placeholder 'e.g. Parol, Augmentin'.
 - GENERIC NAME (OPTIONAL): Input field with placeholder 'e.g. Paracetamol'.
 - FORM: Radio buttons for TABLET, CAPSULE, SYRUP, CREAM. 'TABLET' is selected.
 - DOSAGE / STRENGTH: Input field with placeholder 'e.g. 500 mg'.
 - Bottom Button: Add Medication.
- Screen 2: Schedule**
 - Header: New Medication (with a back arrow).
 - Section: Schedule
 - TIMES PER DAY?: Radio buttons for 1x, 2x, 3x, 4x. '1x' is selected.
 - MEAL INSTRUCTIONS: Radio buttons for Before Meal, After Meal, With Food. 'Before Meal' is selected.
 - FREQUENCY: Radio buttons for Daily, Weekly, As Needed, F. 'Daily' is selected.
 - REMINDER TIMES: Input field with clock icon, '09:00', and a red 'x' icon. A '+ Add Time' link is present.
 - Bottom Button: Add Medication.
- Screen 3: Inventory Tracking and AI Note**
 - Header: New Medication (with a back arrow).
 - Section: Inventory Tracking
 - CURRENT STOCK: Input field with '0'.
 - ALERT LIMIT: Input field with '5'.
 - Section: AI Medication Note
 - Header: AI Medication Note (with a purple icon).
 - Text: Auto-simplifies the drug prospectus.
 - Button: Generate AI Note.
 - Section: NOTES
 - Text: Tap the button above to generate a note...
 - Bottom Button: Add Medication.

Figure 5.4 Add medication manual screens

The cabinet also supports multi-selection for batch deletion. A long press on any item enters selection mode, where the user can tap multiple medications and then delete them all at once. The backend's deleteMany procedure handles this by soft-deleting (setting isActive to false) rather than actually removing the records, so adherence history is preserved.



Figure 5.5: Muliselection of medications

5.4.3 Calendar and History

The calendar screen has a horizontal scrollable date strip showing the last two weeks and the next week. Tapping a date fetches the medication log for that day through the `getDayLog` backend procedure, which returns all active prescriptions along with their intake events and dose schedules for the selected date.

The screen then computes a timeline from this data. For each scheduled dose, it checks whether there is a matching intake event. If there is, the dose is marked as **TAKEN**. If the scheduled time has already passed and there is no event, it is marked as **MISSED**. If the time has not come yet (and it is today), it is marked as **UPCOMING**. The timeline is rendered by a `DailyLogList` component that groups entries chronologically.

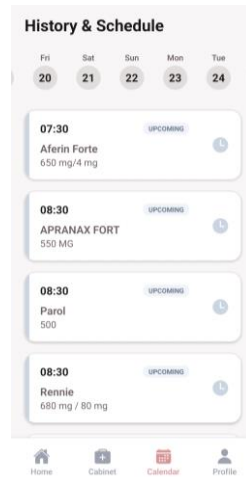


Figure 5.6: Patient calendar tab

5.4.4 Profile and Settings

The profile screen brings together user preferences and accessibility settings. It has sections for:

- Language: A toggle between Turkish and English, which updates the app's locale stored in a `LanguageContext`.
- Theme: A three-option selector for light, dark, or system default, managed through an `AccessibilityContext`.
- Accessibility: A high contrast toggle and a text size slider (0.8x to 2.0x).
- Care Team: A link to the care team management screen.
- Logout.

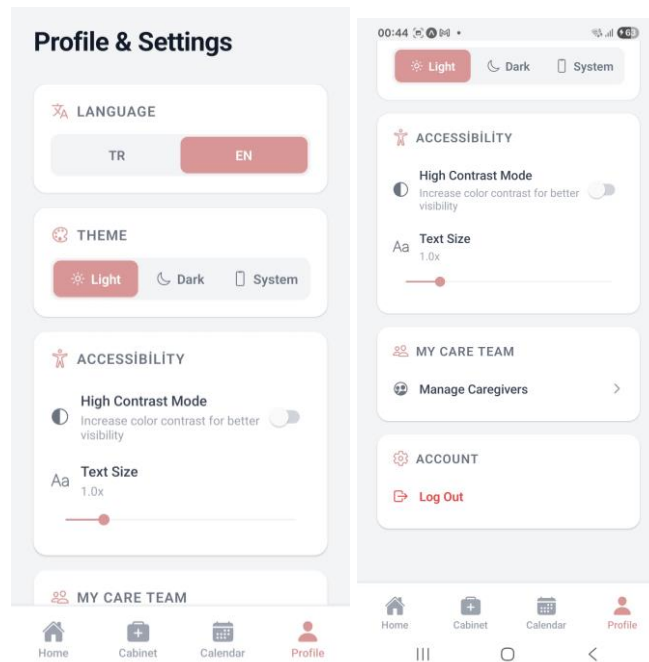


Figure 5.7: Settings page of patient

5.4.5 Care Team Management

From the patient side, the care team screen (accessible from the profile) shows two lists: active caregivers and pending invitations. The patient can invite a caregiver by entering their email address, which creates a CareTeamMember record with status INVITED and initiatedBy set to PATIENT. The patient can also accept invitations from caregivers who have invited them first.

5.5 Caregiver-Side Features

Caregivers have their own tab navigator with four tabs: Home, Patients, Activity Log, and Profile.

5.5.1 Caregiver Dashboard

The caregiver home screen is simpler than the patient's. It shows two stat cards—one for the number of active patients and one for the number of unread activity log entries—and an invite section where the caregiver can send an invitation to a patient by email. Below that, there are quick-action links to navigate to the patients list or the activity log.

The unread alert count is computed by comparing the most recent activity log entry ID against the last-seen ID stored in a Zustand persistent store (more on this in 5.5.3).

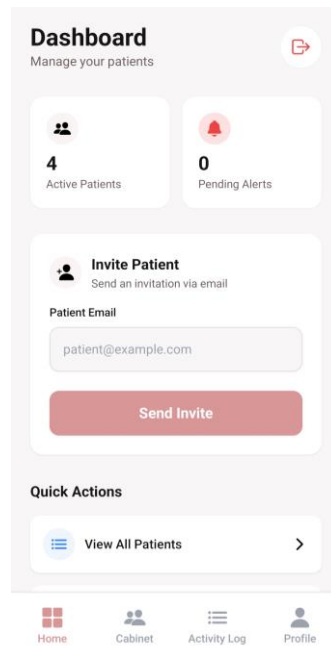


Figure 5.8: Caregiver dashboard page

5.5.2 Patient List

The Patients tab shows all patients with whom the caregiver has an ACTIVE relationship. Each card displays the patient's name and email, with a "View" button that navigates to a patient detail screen. The detail screen calls `getPatientData`, which returns the patient's profile, active medications, and their five most recent intake events per medication—giving the caregiver a quick sense of whether the patient is keeping up with their regimen.

5.5.3 Activity Log

The activity log is probably the most information-dense screen in the app. It fetches all intake events across all of the caregiver's patients (up to the last 100 entries) and renders them as a feed grouped by date.

Each log entry shows the patient's name, the medication name, the intake status (with color-coded icons: green for taken, yellow for skipped, red for missed), and a timing badge. The timing is computed client-side by comparing the event's takenAt timestamp against the medication's scheduled dose times. If the delta is within 30 minutes, it shows "On Time"; otherwise, it shows something like "45 min late" or "2 hrs early."

To handle the "unread" badge on the tab, the app uses a small Zustand store called activityLogStore. This store persists the ID of the most recent log entry that the caregiver has seen (using AsyncStorage under the hood). When the activity log screen mounts, it calls markAsSeen with the newest entry's ID. The tab layout then calls getUnreadCount to compare the seen ID against the current list and show a red badge if there are new entries.

This was one of the trickier parts to get right, because the "unread" state needed to survive app restarts and also update immediately when the caregiver opens the log, without waiting for a network round-trip.

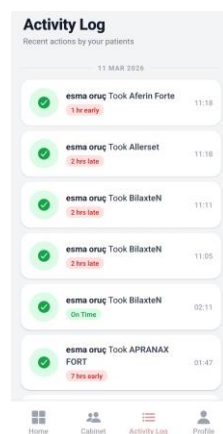


Figure 5.9: Caregiver activity log screen

5.6 AI-Powered Features

MedSafe uses Google's Gemini 2.5 Flash model for two different AI features: generating simplified medication notes and extracting medication data from package photos.

5.6.1 AI Medication Note Generation

The idea behind this feature is that medication prospectuses are usually long, dense, and written in medical jargon—which can be hard for elderly patients to understand. So MedSafe offers a one-tap "Generate Note" button in the medication form that produces a short, plain-language summary.

The pipeline works as follows. First, the backend tries to fetch official drug label information from the OpenFDA Drug Label API. The query searches by both brand name and generic name, with a 6-second timeout to avoid blocking in case the API is slow or down. If it finds a result, it extracts the indications/usage or dosage/administration text.

Then, regardless of whether OpenFDA data was found, the system calls Gemini with a carefully crafted prompt. If FDA data is available, it is included in the prompt as "prospectus information" for the model to summarize. If not, the model is instructed to use its general medical knowledge but to say "Consult your doctor" for anything it is not sure about.

The system prompt enforces a strict format: exactly three bullet points, each at most 20 words, using plain everyday language. The response also has to be in the language the user has selected (Turkish or English), which is specified in the user prompt.

On the mobile side, the useAiNote hook wraps this entire flow. It exposes the generated note, loading and error states, and a function to trigger generation. The user can also edit the AI-generated note before saving.

5.6.2 Camera-Based Medication Scanning

The second AI feature is a camera screen where the patient can photograph a medication box and have the app extract the drug's details automatically. This saves the

patient from having to type in all the information manually, which can be tedious and error-prone—especially for elderly users.

The scan screen uses Expo Camera to capture a photo, which is converted to base64 and sent to the backend's scanBox procedure. The backend creates a Gemini request with the image and a detailed prompt that instructs the model to extract specific fields: generic name, brand name, dosage, form, frequency, daily dose count, meal instructions, and a confidence score.



Figure 5.10: Add medication via OCR

An important aspect is that the prompt is designed to handle Turkish medication packaging specifically. For example, it knows that "aç karnına" means "before meal" and that "günde 3 kez" or "3x1" means three doses per day. The model is also told to return null for any field it cannot determine, rather than guessing.

The response comes back as JSON, which is validated through a Zod schema to ensure it matches the expected structure. If validation fails (for example, because the photo was blurry and the model returned garbled text), the user gets an error message asking them to try again with a clearer photo.

Once the scan succeeds, the results are passed as URL parameters to the add-medication form, where the fields are pre-filled. The user can review and correct anything the AI got wrong before saving.

5.7 Internationalization and Accessibility

Since the target audience for MedSafe includes elderly patients in Turkey, the app needs to work well in both Turkish and English, and it also needs to be usable by people with varying degrees of visual ability.

5.7.1 Language Support

The internationalization system is a straightforward custom implementation. There are two translation files—`en.ts` and `tr.ts`—that export objects with roughly 200 keys covering all user-facing text in the application. A `LanguageContext` wraps the app and provides a `t()` function that looks up translations by key, along with a locale value and a `setLocale` function.

The selected language is persisted using `AsyncStorage` so it survives app restarts. The language setting also affects the AI note generation: when a patient generates a medication note, the app sends the current locale to the backend, and the Gemini prompt is adjusted to produce the note in the corresponding language.

One area where this was particularly important is the medication scanning feature. The scan prompt already includes Turkish-specific patterns for medication packaging text, so the scanner works with both Turkish and English packaging regardless of the app's language setting.

5.7.2 Accessibility

The accessibility system is managed through an `AccessibilityContext` that provides three settings:

High contrast mode switches the entire color scheme to use bold black text on white backgrounds with heavy 2px borders around all interactive elements. This makes the UI much easier to read for users with reduced vision.

Adjustable text size allows users to scale all text in the app from 0.8x to 2.0x using a slider on the profile screen. This is implemented by passing a `textSize` multiplier to style factory functions—every screen has a `makeStyles(isHighContrast, isDarkMode, textSize)` function that generates a `StyleSheet` with all font sizes multiplied by the current setting.

Theme selection lets users choose between light mode, dark mode, or following the system preference. Like the language setting, themes are persisted through `AsyncStorage`.

These settings are consistent across all patient screens. The accessibility context values are consumed by every screen component to ensure the entire app responds to changes immediately, without needing to restart.

REFERENCES

Brown, R., & Davis, L. (2025). Voice Usability for Older Adults: A Comprehensive Study. *International Journal of Human-Computer Interaction*, 41(3), 200-215.

- Grindrod, K. A., Li, M., & Gates, A. (2014). Evaluating User Perceptions of Mobile Medication Management Applications With Older Adults: A Usability Study. *JMIR mHealth and uHealth*, 2(1), e11. <https://doi.org/10.2196/mhealth.3048>
- Liang, F., Wang, Y., & Zhang, L. (2023). Medication adherence management for in-home geriatric care: A systematic review. *JMIR Formative Research*, 7(1), e12345.
- Liu, M., Wang, C., & Hu, J. (2023). Older adults' intention to use voice assistants: Usability and emotional needs. *Heliyon*, 9(11), e21932. <https://doi.org/10.1016/j.heliyon.2023.e21932>
- Mahmood, A., Cao, S., Stiber, M., Antony, V., & Huang, C. M. (2025). Voice Assistants for Health Self-Management: Designing for and with Older Adults. *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 1-22. <https://doi.org/10.1145/3706598.3713839>
- MedScan Research Team. (2025). MedScan: An AI-Powered Mobile App Designed to Identify Medications for People with Visual Impairments. *ResearchGate Preprint*.
- Paul, S., Gupta, A., & Lee, K. (2024). Smart Medicine Reminder Kit with Health Monitoring using IoT. *Procedia Computer Science*, 230, 45-52.
- Shade, M., Rector, K., & Kupzyk, K. (2021). Voice Assistant reminders and the latency of scheduled medication use in Older Adults with Pain: Descriptive Feasibility study. *JMIR Formative Research*, 5(9), e26361. <https://doi.org/10.2196/26361>
- Smith, A., & Johnson, B. (2025). AI in Managing Polypharmacy – Scoping Review. *Journal of Artificial Intelligence in Medicine*, 12(2), 100-112.
- Sokol, M. C., McGuigan, K. A., Verbrugge, R. R., & Epstein, R. S. (2005). Impact of medication adherence on hospitalization risk and healthcare cost. *Medical Care*, 43(6), 521–530.

- SPEC 2.0 Study Group. (2023). SPEC 2.0: IoT Based Smart Medication Management System for Elderly. *Springer Journal of Medical Systems*, 42(3).
- Stuck, R. E., Chong, A. W., Tracy, L. M., & Rogers, W. A. (2017). Medication management apps: usable by older adults? *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, 61(1), 1141-1144.
<https://doi.org/10.1177/1541931213601769>
- Vergara-Villegas, O., Martinez, S., & Lopez, A. (2025). AR + Automatic Pillbox for Medication Adherence: A Pilot Study. *Journal of Biomedical Informatics*, 142, 104356.
- Web Content Accessibility Guidelines (WCAG) 2.1. (2023, September 21). *W3C Recommendation*. <https://www.w3.org/TR/WCAG21/>
- Wilson, J., et al. (2007). Medication adherence and clinical outcomes in the elderly. *Archives of Internal Medicine*, 167(8), 810-815.